

Este projeto consiste em uma API REST desenvolvida com Node.js e Express, projetada para demonstrar a integração entre bancos de dados relacionais e não relacionais em uma mesma aplicação. Para o armazenamento de dados relacionais, foi utilizado PostgreSQL em conjunto com o Prisma ORM, responsável pelo gerenciamento dos usuários e das informações que exigem maior consistência transacional. Já os recursos de natureza NoSQL, como carros, motos e marcas de roupa, são persistidos em MongoDB por meio do Mongoose, aproveitando a flexibilidade oferecida pelo modelo orientado a documentos.

A aplicação implementa um sistema completo de autenticação e autorização baseado em JSON Web Tokens (JWT), permitindo autenticação stateless e controle de acesso eficiente. As senhas dos usuários são armazenadas de forma segura utilizando bcrypt para geração e verificação de hashes. Além disso, a autorização é realizada por meio de papéis (roles), diferenciando usuários comuns (USER) e administradores (ADMIN), garantindo que cada perfil possua acesso apenas aos recursos permitidos.

Com foco em segurança, foram adotadas diversas práticas recomendadas pelas diretrizes da OWASP. Entre elas destacam-se a utilização do Helmet para configuração de cabeçalhos HTTP seguros, o CORS para controle de acesso entre origens, o Express Rate Limit para mitigação de ataques de força bruta e abuso de requisições, além da validação rigorosa dos dados recebidos pela API. O tratamento centralizado de erros também foi implementado para garantir respostas consistentes e evitar a exposição de informações sensíveis.

A qualidade da aplicação é reforçada por uma suíte de testes automatizados desenvolvida com Jest e Supertest. Os testes contemplam os principais fluxos e endpoints da API, validando autenticação, autorização, operações CRUD e regras de negócio. Todos os testes foram configurados para execução tanto em containers Docker, garantindo reprodutibilidade e confiabilidade durante o desenvolvimento e a integração contínua.

Para facilitar a implantação e a execução do sistema, o projeto inclui um Dockerfile e um arquivo docker-compose.yml responsáveis pela orquestração dos serviços da API, PostgreSQL e MongoDB. Dessa forma toda a infraestrutura necessária pode ser inicializada por meio do comando `docker compose up -- build`, simplificando o processo de configuração do ambiente.

A documentação da API é gerada automaticamente utilizando Swagger por meio das bibliotecas swagger-jsdoc e swagger-ui-express. A interface interativa de documentação fica disponível no endpoint `/api-docs`, permitindo visualizar, testar e compreender facilmente todos os recursos disponibilizados pela aplicação.

A estrutura do projeto foi organizada seguindo princípios de separação de responsabilidades, distribuindo o código em camadas como controllers, services, models, routes, middlewares, schemas e testes. Essa organização favorece a manutenção, escalabilidade, reutilização de código e desenvolvimento orientado a testes.

As configurações da aplicação são realizadas por meio de variáveis de ambiente, cujos exemplos são disponibilizados no arquivo `.env.example`. Por questões de segurança, o arquivo `.env` contendo credenciais reais não deve ser versionado. Entre as principais variáveis utilizadas estão `DATABASE_URL`, responsável pela conexão com o PostgreSQL, `MONGO_URI` para acesso ao MongoDB, `JWT_SECRET` para assinatura dos tokens de autenticação e `PORT` para definição da porta de execução da aplicação.

As principais decisões técnicas adotadas neste projeto foram motivadas por critérios de produtividade, manutenção e escalabilidade. O Prisma foi escolhido pela facilidade na criação e gerenciamento de migrações, além da forte tipagem oferecida ao desenvolvimento. O Mongoose foi selecionado pela flexibilidade na modelagem de documentos em MongoDB. A utilização de JWT possibilita autenticação desacoplada do servidor, enquanto a separação entre controllers e services contribui para maior testabilidade e organização da lógica de negócios.

Para executar o projeto, basta clonar o repositório, criar um arquivo `.env` com base no `.env.example`, iniciar os serviços utilizando `docker compose up --build`, acessar a documentação disponível em <http://localhost:3000/api-docs> e, por fim, executar os testes por meio do comando `npm test`, seja localmente ou dentro do container, para validar o correto funcionamento da aplicação.